

=====

BIBLE CRYPTOGRAPHY – RÉFÉRENCE EXHAUSTIVE DU MODULE PYTHON

LÉGENDE : [sym] = chiffrement symétrique | [asym] = asymétrique | [h] = hachage
[x509] = certificats | [util] = utilitaires

=====

[util] Génération de nombres aléatoires sécurisés – os.urandom / token_bytes |

DESCRIPTION : Génère des octets ou tokens aléatoires cryptographiquement sûrs
POURQUOI : Base de toute cryptographie sécurisée. Un aléa prévisible = vulnérabilité.
CONTEXTE : Génération de clés, IV, nonces, sel (salt), tokens de session.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives import constant_time
import os
```

os.urandom(16)	Génère 16 octets aléatoires sécurisés
os.urandom(32)	Génère 32 octets (clé AES-256)
os.urandom(12)	Génère 12 octets (nonce GCM)

```
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
# → voir section KDF pour usage du sel
```

EXEMPLES :

key = os.urandom(32)	# Clé AES-256 bits
iv = os.urandom(16)	# IV pour CBC
nonce = os.urandom(12)	# Nonce pour GCM
salt = os.urandom(16)	# Sel pour KDF

[sym] Chiffrement symétrique Fernet – cryptography.fernet |

DESCRIPTION : Chiffrement symétrique authentifié basé sur AES-128-CBC + HMAC-SHA256
POURQUOI : API de haut niveau, simple et sûre par défaut. Inclut intégrité + authenticité.
CONTEXTE : Chiffrement de données applicatives, tokens, secrets de configuration.

FONCTIONS PRINCIPALES :

```
from cryptography.fernet import Fernet, MultiFernet
```

— CLÉS —

Fernet.generate_key()	Génère une clé Fernet (32 bytes, encodée Base64)
key = Fernet.generate_key()	Retourne un objet bytes URL-safe Base64
f = Fernet(key)	Instancie un objet Fernet avec la clé

— CHIFFREMENT —

f.encrypt(b"message")	Chiffre des bytes → bytes chiffrés
f.encrypt_at_time(data, current_time)	Chiffre avec timestamp personnalisé

— DÉCHIFFREMENT —

f.decrypt(token)	Déchiffre un token Fernet → bytes
f.decrypt_at_time(token, ttl, current_time)	Déchiffre avec contrôle TTL et timestamp
f.extract_timestamp(token)	Extrait le timestamp du token (int UNIX)

— ROTATION DE CLÉS (MultiFernet) —

keys = [Fernet(k1), Fernet(k2)]	
mf = MultiFernet(keys)	Essaie chaque clé dans l'ordre
mf.encrypt(b"message")	Chiffre avec la première clé
mf.decrypt(token)	Déchiffre avec la première clé valide
mf.rotate(token)	Re-chiffre un token avec la nouvelle clé

EXEMPLES :

```
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"données sensibles")
print(f.decrypt(token)) # b'données sensibles'
```

```
# Rotation de clé
old_key = Fernet.generate_key()
new_key = Fernet.generate_key()
mf = MultiFernet([Fernet(new_key), Fernet(old_key)])
token_migre = mf.rotate(ancien_token)
```

DESCRIPTION : Chiffrement AES avec contrôle total du mode opératoire et du padding
 POURQUOI : Nécessaire quand on doit interopérer avec d'autres systèmes ou choisir précisément le mode (CBC, GCM, CTR, CFB, OFB...).

CONTEXTE : Protocoles réseau, interopérabilité, chiffrement de fichiers volumineux.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sym_padding
from cryptography.hazmat.backends import default_backend
```

— ALGORITHMES —

algorithms.AES(key)	AES (clé 16, 24 ou 32 octets)
algorithms.AES128(key)	AES-128 (clé 16 octets)
algorithms.AES256(key)	AES-256 (clé 32 octets)
algorithms.ChaCha20(key, nonce)	ChaCha20 (clé 32 octets, nonce 16 octets)
algorithms.TripleDES(key)	3DES (clé 8, 16 ou 24 octets) – legacy
algorithms.Blowfish(key)	Blowfish – legacy
algorithms.CAST5(key)	CAST5 – legacy
algorithms.SEED(key)	SEED – legacy

— MODES OPÉRATOIRES —

modes.CBC(iv)	Cipher Block Chaining (IV 16 octets)
modes.CTR(nonce)	Counter Mode (nonce 16 octets)
modes.CFB(iv)	Cipher Feedback (IV 16 octets)
modes.CFB8(iv)	CFB avec segment de 8 bits
modes.OFB(iv)	Output Feedback (IV 16 octets)
modes.ECB()	Electronic Codebook – déconseillé
modes.GCM(nonce)	Galois/Counter Mode (authentifié)
modes.GCM(nonce, tag=tag)	GCM avec tag pour déchiffrement
modes.GCM(nonce, min_tag_length=16)	Longueur minimale du tag GCM
modes.XTS(tweak)	XTS pour chiffrement de disques
modes.SIV(nonce=None)	SIV (nonce synthétique)

— CRÉATION ET USAGE DU CIPHER —

```
cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
encryptor = cipher.encryptor()           Crée un objet chiffreur
decryptor = cipher.decryptor()           Crée un objet déchiffreur

encryptor.update(data)                    Chiffre un bloc de données
encryptor.finalize()                      Finalise le chiffrement
decryptor.update(ciphertext)              Déchiffre un bloc
decryptor.finalize()                      Finalise le déchiffrement
```

— PADDING —

sym_padding.PKCS7(128).padder()	Padder PKCS7 pour blocs 128 bits (AES)
padder.update(data)	Ajoute les données à padder
padder.finalize()	Retourne les données paddées
sym_padding.PKCS7(128).unpadder()	Unpadder pour retirer le padding
sym_padding.ANSIX923(128).padder()	Padding ANSI X.923

— CHIFFREMENT AUTHENTIFIÉ (GCM) —

```
cipher = Cipher(algorithms.AES(key), modes.GCM(nonce))
encryptor = cipher.encryptor()
encryptor.authenticate_additional_data(aad)  Authentifie des données non chiffrées
ct = encryptor.update(plaintext) + encryptor.finalize()
tag = encryptor.tag                          Tag d'authentification GCM (16 octets)
```

EXEMPLES :

```
# AES-CBC avec PKCS7
key = os.urandom(32)
iv = os.urandom(16)
padder = sym_padding.PKCS7(128).padder()
padded = padder.update(b"message") + padder.finalize()
cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
ct = cipher.encryptor().update(padded) + cipher.encryptor().finalize()
```

```
# AES-GCM (recommandé)
key = os.urandom(32)
nonce = os.urandom(12)
cipher = Cipher(algorithms.AES(key), modes.GCM(nonce))
enc = cipher.encryptor()
ct = enc.update(b"message") + enc.finalize()
```

tag = enc.tag

[h] Hachage – hazmat.primitives.hashes

DESCRIPTION : Calcule une empreinte cryptographique d'un message (one-way function)
POURQUOI : Vérification d'intégrité, stockage de mots de passe (avec KDF), signatures.
CONTEXTE : Intégrité de fichiers, authentification, protocoles TLS, blockchain.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.hashes import Hash
```

— ALGORITHMES DE HACHAGE —

hashes.SHA256()	SHA-256 (32 octets) – recommandé
hashes.SHA512()	SHA-512 (64 octets)
hashes.SHA384()	SHA-384 (48 octets)
hashes.SHA224()	SHA-224 (28 octets)
hashes.SHA1()	SHA-1 – déprécié, éviter
hashes.SHA3_256()	SHA-3/256 (32 octets)
hashes.SHA3_512()	SHA-3/512 (64 octets)
hashes.SHA3_384()	SHA-3/384
hashes.SHA3_224()	SHA-3/224
hashes.BLAKE2b(digest_size=64)	BLAKE2b (1-64 octets)
hashes.BLAKE2s(digest_size=32)	BLAKE2s (1-32 octets)
hashes.MD5()	MD5 – déprécié, éviter

— USAGE —

digest = Hash(hashes.SHA256())	Crée un objet de hachage
digest.update(b"données")	Ajoute des données
digest.update(b"suite")	Peut être appelé plusieurs fois
digest.finalize()	Retourne le hash bytes (détruit l'objet)
digest.copy()	Copie l'état courant du hash

— PROPRIÉTÉS —

hashes.SHA256().digest_size	Taille du hash en octets (32)
hashes.SHA256().block_size	Taille de bloc interne en octets (64)
hashes.SHA256().name	Nom de l'algorithme ("sha256")

EXEMPLES :

```
from cryptography.hazmat.primitives.hashes import Hash, SHA256
h = Hash(SHA256())
h.update(b"bonjour")
h.update(b" monde")
digest = h.finalize()
print(digest.hex()) # Empreinte SHA-256 en hexadécimal

# Hash en une ligne avec hashlib (interopérable)
import hashlib
hashlib.sha256(b"message").hexdigest()
```

[h] HMAC – codes d'authentification de message – hazmat.primitives.hmac

DESCRIPTION : Calcule un MAC (Message Authentication Code) à clé secrète
POURQUOI : Authentifie l'origine et l'intégrité d'un message sans chiffrement.
CONTEXTE : APIs REST, cookies signés, vérification de webhooks, protocoles réseau.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives import hmac, hashes
```

hmac.HMAC(key, hashes.SHA256())	Crée un objet HMAC
h = hmac.HMAC(key, hashes.SHA256())	
h.update(b"message")	Ajoute des données
h.finalize()	Retourne le MAC (bytes)
h.verify(signature)	Vérifie le MAC (lève InvalidSignature)
h.copy()	Copie l'état courant

EXEMPLES :

```
from cryptography.hazmat.primitives import hmac, hashes
key = os.urandom(32)
h = hmac.HMAC(key, hashes.SHA256())
h.update(b"données à authentifier")
```

```

mac = h.finalize()

# Vérification
h2 = hmac.HMAC(key, hashes.SHA256())
h2.update(b"données à authentifier")
h2.verify(mac) # OK si identique, sinon InvalidSignature

```

[h] Fonctions de dérivation de clé (KDF) – hazmat.primitives.kdf

DESCRIPTION : Dérive une clé cryptographique depuis un mot de passe ou un secret
POURQUOI : Un mot de passe ne peut pas être utilisé directement comme clé.
Les KDF ajoutent sel, itérations et coût mémoire contre le bruteforce.
CONTEXTE : Stockage de mots de passe, dérivation de clés de session, PAKE.

FONCTIONS PRINCIPALES :

```

from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from cryptography.hazmat.primitives.kdf.hkdf import HKDF, HKDFExpand
from cryptography.hazmat.primitives.kdf.x963kdf import X963KDF
from cryptography.hazmat.primitives.kdf.concatkdf import ConcatKDFHash, ConcatKDFHMAC
from cryptography.hazmat.primitives import hashes

```

PBKDF2

```

PBKDF2HMAC(
    algorithm=hashes.SHA256(),
    length=32,
    salt=salt,
    iterations=480000
)
kdf.derive(password)
kdf.verify(password, expected_key)

```

Dérivation basée sur HMAC itéré
Longueur de la clé dérivée (octets)
Sel aléatoire (min 16 octets)
Nombre d'itérations (NIST 2023: 600 000)
Dérive la clé depuis le mot de passe (bytes)
Vérifie sans révéler la clé

SCRYPT

```

Scrypt(
    salt=salt,
    length=32,
    n=2*14,
    r=8,
    p=1
)
kdf.derive(b"mot_de_passe")
kdf.verify(b"mot_de_passe", expected_key)

```

KDF résistant au matériel spécialisé
Paramètre CPU/mémoire (puissance de 2)
Taille de bloc
Parallélisme
Dérive la clé
Vérifie

HKDF

```

HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=salt,
    info=b"contexte applicatif"
)
kdf.derive(input_key_material)

HKDFExpand(
    algorithm=hashes.SHA256(),
    length=32,
    info=b"contexte"
)
kdf.derive(prk)

```

Dérivation depuis un secret uniforme
Optionnel (None accepté)
Contexte (optionnel mais recommandé)
Dérive une clé depuis un IKM
Phase "expand" seulement
Dérive depuis une pseudo-random key

EXEMPLES :

```

import base64, os
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes
from cryptography.fernet import Fernet

salt = os.urandom(16)
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt, iterations=480000)
key = base64.urlsafe_b64encode(kdf.derive(b"mon_mot_de_passe"))
f = Fernet(key)
token = f.encrypt(b"secret")

```

DESCRIPTION : Génération, sérialisation et usage de clés RSA
 POURQUOI : Standard de facto pour le chiffrement asymétrique et les signatures.
 CONTEXTE : TLS, signatures numériques, échange de clés, certificats X.509.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding as asym_padding
from cryptography.hazmat.primitives import serialization, hashes
```

— GÉNÉRATION DE CLÉS —

```
rsa.generate_private_key(          Génère une paire de clés RSA
    public_exponent=65537,         Exposant public (65537 recommandé)
    key_size=2048                  Taille en bits (2048 min, 4096 recommandé)
)
private_key.public_key()           Extrait la clé publique
```

— PROPRIÉTÉS DE LA CLÉ —

```
private_key.key_size               Taille de la clé en bits
private_key.public_key().key_size  Taille de la clé publique
private_key.private_numbers()      Composantes numériques privées
public_key.public_numbers()        Composantes numériques publiques
public_key.public_numbers().e      Exposant public
public_key.public_numbers().n      Module public
```

— CHIFFREMENT / DÉCHIFFREMENT RSA —

```
public_key.encrypt(message, asym_padding.OAEP(  Chiffrement OAEP (recommandé)
    mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
    algorithm=hashes.SHA256(),
    label=None
))
private_key.decrypt(ciphertext, asym_padding.OAEP(  Déchiffrement OAEP
    mgf=asym_padding.MGF1(algorithm=hashes.SHA256()),
    algorithm=hashes.SHA256(),
    label=None
))
public_key.encrypt(msg, asym_padding.PKCS1v15())  Chiffrement PKCS1v1.5 – legacy
private_key.decrypt(ct, asym_padding.PKCS1v15())  Déchiffrement PKCS1v1.5 – legacy
```

— SIGNATURE / VÉRIFICATION RSA —

```
private_key.sign(message, asym_padding.PSS(      Signature PSS (recommandée)
    mgf=asym_padding.MGF1(hashes.SHA256()),
    salt_length=asym_padding.PSS.MAX_LENGTH
), hashes.SHA256())
public_key.verify(signature, message, asym_padding.PSS(  Vérification PSS
    mgf=asym_padding.MGF1(hashes.SHA256()),
    salt_length=asym_padding.PSS.MAX_LENGTH
), hashes.SHA256())                                     Lève InvalidSignature si invalide
private_key.sign(msg, asym_padding.PKCS1v15(), hashes.SHA256())  Signature PKCS1v1.5
public_key.verify(sig, msg, asym_padding.PKCS1v15(), hashes.SHA256())
```

— SÉRIALISATION —

```
private_key.private_bytes(          Sérialise la clé privée
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.BestAvailableEncryption(b"passphrase")
)
private_key.private_bytes(          Clé privée non chiffrée
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.TraditionalOpenSSL,
    encryption_algorithm=serialization.NoEncryption()
)
public_key.public_bytes(            Sérialise la clé publique
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
serialization.Encoding.PEM         Format PEM (Base64 + en-têtes)
serialization.Encoding.DER         Format DER (binaire)
```

— CHARGEMENT DE CLÉS —

```
serialization.load_pem_private_key(pem_data, password=None)  Charge clé privée PEM
serialization.load_pem_private_key(pem_data, password=b"pass")  Avec mot de passe
serialization.load_pem_public_key(pem_data)                  Charge clé publique PEM
serialization.load_der_private_key(der_data, password=None)   Charge clé privée DER
serialization.load_der_public_key(der_data)                   Charge clé publique DER
```

EXEMPLES :

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding as p
from cryptography.hazmat.primitives import hashes, serialization

priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
pub = priv.public_key()

ct = pub.encrypt(b"secret", p.OAEP(
    mgf=p.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None))
msg = priv.decrypt(ct, p.OAEP(
    mgf=p.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None))

pem = priv.private_bytes(serialization.Encoding.PEM,
    serialization.PrivateFormat.PKCS8, serialization.NoEncryption())
```

[asym] Clés Elliptiques (EC) – hazmat.primitives.asymmetric.ec
--

DESCRIPTION : Clés elliptiques pour signatures (ECDSA) et échange de clés (ECDH)
POURQUOI : Sécurité équivalente à RSA avec des clés beaucoup plus courtes.
CONTEXTE : TLS moderne, signatures JWT, échange Diffie-Hellman sur courbe elliptique.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives.asymmetric import ec
```

— COURBES DISPONIBLES —

ec.SECP256R1()	P-256 (NIST) – très répandu
ec.SECP384R1()	P-384 (NIST) – haute sécurité
ec.SECP521R1()	P-521 (NIST) – très haute sécurité
ec.SECP256K1()	secp256k1 – Bitcoin/Ethereum
ec.BrainpoolP256R1()	Brainpool P-256
ec.BrainpoolP384R1()	Brainpool P-384
ec.BrainpoolP512R1()	Brainpool P-512

— GÉNÉRATION DE CLÉS —

ec.generate_private_key(ec.SECP256R1())	Génère une clé privée EC
private_key.public_key()	Extrait la clé publique
private_key.key_size	Taille de la clé en bits
private_key.curve	Courbe utilisée
private_key.private_numbers()	Composantes numériques
public_key.public_numbers()	Composantes numériques publiques

— SIGNATURE / VÉRIFICATION ECDSA —

private_key.sign(data, ec.ECDSA(hashes.SHA256()))	Signe avec ECDSA
public_key.verify(sig, data, ec.ECDSA(hashes.SHA256()))	Vérifie (lève InvalidSignature)
private_key.sign(data, ec.ECDSA(hashes.SHA384()))	Signature avec SHA-384
private_key.sign(data, ec.ECDSA(utils.Prehashed()))	Données déjà hachées

— ÉCHANGE DE CLÉS ECDH —

```
from cryptography.hazmat.primitives.asymmetric.ec import ECDH
shared_key = private_key.exchange(ECDH(), peer_public_key) Dérive le secret partagé
```

— SÉRIALISATION —

```
private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)
public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
# Formats de point de courbe publique
serialization.PublicFormat.UncompressedPoint  Format non compressé (04 || x || y)
serialization.PublicFormat.CompressedPoint    Format compressé (02/03 || x)
```

EXEMPLES :

```
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives import hashes

priv = ec.generate_private_key(ec.SECP256R1())
pub = priv.public_key()
sig = priv.sign(b"message à signer", ec.ECDSA(hashes.SHA256()))
```

```
pub.verify(sig, b"message à signer", ec.ECDSA(hashes.SHA256())) # OK ou exception
```

```
# ECDH
alice_priv = ec.generate_private_key(ec.SECP256R1())
bob_priv   = ec.generate_private_key(ec.SECP256R1())
shared_alice = alice_priv.exchange(ec.ECDH(), bob_priv.public_key())
shared_bob   = bob_priv.exchange(ec.ECDH(), alice_priv.public_key())
# shared_alice == shared_bob
```

```
[asym] Clés Ed25519 / X25519 – hazmat.primitives.asymmetric.ed25519 / x25519 |
```

DESCRIPTION : Courbes modernes de Bernstein – plus rapides et plus simples que EC classique
POURQUOI : Moins de pièges d'implémentation, performances excellentes.
CONTEXTE : SSH moderne, TLS 1.3, Signal Protocol, WireGuard.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
```

— Ed25519 (signatures) —

Ed25519PrivateKey.generate()	Génère une clé privée Ed25519
private_key.public_key()	Extrait la clé publique
private_key.sign(data)	Signe des données (bytes) → 64 octets
public_key.verify(signature, data)	Vérifie (lève InvalidSignature si invalide)
private_key.private_bytes_raw()	Clé privée brute (32 octets)
public_key.public_bytes_raw()	Clé publique brute (32 octets)

— Ed448 (signatures haute sécurité) —

from cryptography.hazmat.primitives.asymmetric.ed448 import Ed448PrivateKey	
Ed448PrivateKey.generate()	Génère une clé privée Ed448
private_key.sign(data)	Signe (114 octets de signature)
public_key.verify(signature, data)	Vérifie

— X25519 (échange Diffie-Hellman) —

X25519PrivateKey.generate()	Génère une clé privée X25519
private_key.public_key()	Clé publique associée
private_key.exchange(peer_public_key)	Dérive le secret partagé (32 octets)
private_key.private_bytes_raw()	Clé privée brute (32 octets)
public_key.public_bytes_raw()	Clé publique brute (32 octets)

— X448 (échange haute sécurité) —

from cryptography.hazmat.primitives.asymmetric.x448 import X448PrivateKey	
X448PrivateKey.generate()	Génère une clé X448
private_key.exchange(peer_public_key)	Dérive le secret partagé (56 octets)

EXEMPLES :

```
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
priv = Ed25519PrivateKey.generate()
sig = priv.sign(b"message")
priv.public_key().verify(sig, b"message") # OK ou InvalidSignature
```

```
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
alice = X25519PrivateKey.generate()
bob = X25519PrivateKey.generate()
shared_alice = alice.exchange(bob.public_key())
shared_bob = bob.exchange(alice.public_key())
# shared_alice == shared_bob (32 octets)
```

```
[asym] Diffie-Hellman (DH) – hazmat.primitives.asymmetric.dh |
```

DESCRIPTION : Échange de clés Diffie-Hellman sur groupe fini
POURQUOI : Permet d'établir un secret partagé sur un canal non sécurisé.
CONTEXTE : TLS, SSH, établissement de sessions chiffrées.

FONCTIONS PRINCIPALES :

```
from cryptography.hazmat.primitives.asymmetric import dh
```

dh.generate_parameters(generator=2, key_size=2048)	Génère les paramètres DH
parameters.generate_private_key()	Génère une clé privée DH
private_key.public_key()	Extrait la clé publique
private_key.exchange(peer_public_key)	Dérive le secret partagé (bytes)

```

parameters.parameter_numbers()      Composantes p, g, q
parameters.parameter_bytes(         S rialise les param tres
    encoding=serialization.Encoding.PEM,
    format=serialization.ParameterFormat.PKCS3
)
serialization.load_pem_parameters(pem_data) Charge des param tres PEM

```

EXEMPLES :

```

from cryptography.hazmat.primitives.asymmetric import dh
params = dh.generate_parameters(generator=2, key_size=2048)
alice = params.generate_private_key()
bob = params.generate_private_key()
shared_a = alice.exchange(bob.public_key())
shared_b = bob.exchange(alice.public_key())
# shared_a == shared_b → passer dans HKDF pour d river une cl  AES

```

[x509] Certificats X.509 – cryptography.x509
--

DESCRIPTION : Cr e, lit et v rifie des certificats X.509 (format standard TLS/PKI)
POURQUOI : Infrastructure   cl  publique (PKI), TLS, authentification mutuelle.
CONTEXTE : Serveurs HTTPS, VPN, authentification de clients, autorit s de certification.

FONCTIONS PRINCIPALES :

```

from cryptography import x509
from cryptography.x509.oid import NameOID, ExtendedKeyUsageOID
from cryptography.hazmat.primitives import hashes, serialization
import datetime

```

— CHARGEMENT —

x509.load_pem_x509_certificate(pem_data)	Charge un certificat PEM
x509.load_der_x509_certificate(der_data)	Charge un certificat DER
x509.load_pem_x509_csr(pem_data)	Charge une CSR PEM
x509.load_der_x509_csr(der_data)	Charge une CSR DER

— INSPECTION D'UN CERTIFICAT —

cert.subject	Sujet du certificat (objet Name)
cert.issuer	�metteur
cert.serial_number	Num�ro de s�rie
cert.not_valid_before_utc	Date de d�but de validit� (UTC)
cert.not_valid_after_utc	Date de fin de validit� (UTC)
cert.public_key()	Cl� publique du certificat
cert.signature	Signature bytes
cert.signature_hash_algorithm	Algorithme de hachage de signature
cert.fingerprint(hashes.SHA256())	Empreinte du certificat
cert.version	Version X.509 (x509.Version.v3)
cert.extensions	Extensions X.509
cert.extensions.get_extension_for_oid(oid)	R�cup�re une extension par OID
cert.public_bytes(serialization.Encoding.PEM)	S�rialise en PEM

— NOMS (Name) —

x509.Name([	Construit un sujet/�metteur
x509.NameAttribute(NameOID.COUNTRY_NAME, "FR"),	
x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, "�le-de-France"),	
x509.NameAttribute(NameOID.LOCALITY_NAME, "Paris"),	
x509.NameAttribute(NameOID.ORGANIZATION_NAME, "MaCorp"),	
x509.NameAttribute(NameOID.COMMON_NAME, "exemple.com"),	
])	
NameOID.COMMON_NAME	CN
NameOID.ORGANIZATION_NAME	O
NameOID.ORGANIZATIONAL_UNIT_NAME	OU
NameOID.COUNTRY_NAME	C
NameOID.STATE_OR_PROVINCE_NAME	ST
NameOID.LOCALITY_NAME	L
NameOID.EMAIL_ADDRESS	emailAddress

— CR ATION D'UN CERTIFICAT AUTO-SIGN  —

x509.CertificateBuilder()	D�marre un builder de certificat
.subject_name(name)	D�finit le sujet
.issuer_name(name)	D�finit l'�metteur
.public_key(public_key)	Cl� publique � certifier
.serial_number(x509.random_serial_number())	Num�ro de s�rie al�atoire
.not_valid_before(datetime.datetime.utcnow())	D�but de validit�
.not_valid_after(datetime.datetime.utcnow() + datetime.timedelta(days=365))	

<code>.add_extension(ext, critical=False)</code>	Ajoute une extension
<code>.sign(private_key, hashes.SHA256())</code>	Signe le certificat → retourne le cert

— EXTENSIONS COURANTES —

<code>x509.SubjectAlternativeName([</code>	SAN (noms alternatifs)
<code> x509.DNSName("exemple.com"),</code>	
<code> x509.DNSName("*.exemple.com"),</code>	
<code> x509.IPAddress(ipaddress.IPv4Address("192.168.1.1")),</code>	
<code> x509.RFC822Name("admin@exemple.com"),</code>	
<code>])</code>	
<code>x509.BasicConstraints(ca=False, path_length=None)</code>	Certificat final
<code>x509.BasicConstraints(ca=True, path_length=0)</code>	CA intermédiaire
<code>x509.KeyUsage(digital_signature=True, ...)</code>	Usages de la clé
<code>x509.ExtendedKeyUsage([ExtendedKeyUsageOID.SERVER_AUTH])</code>	Usage étendu
<code>x509.SubjectKeyIdentifier.from_public_key(pub)</code>	SKI depuis la clé publique

— CRÉATION D'UNE CSR (Certificate Signing Request) —

<code>x509.CertificateSigningRequestBuilder()</code>	Démarre un builder de CSR
<code>.subject_name(name)</code>	Sujet de la CSR
<code>.add_extension(ext, critical=False)</code>	Extensions demandées
<code>.sign(private_key, hashes.SHA256())</code>	Signe → retourne la CSR

EXEMPLES :

```
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa
import datetime

key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
name = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, "exemple.com")])
cert = (x509.CertificateBuilder()
        .subject_name(name).issuer_name(name)
        .public_key(key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.datetime.utcnow())
        .not_valid_after(datetime.datetime.utcnow() + datetime.timedelta(365))
        .sign(key, hashes.SHA256()))
pem = cert.public_bytes(serialization.Encoding.PEM)
```

[util] Comparaison à temps constant – cryptography.hazmat.primitives

DESCRIPTION : Compare deux séquences d'octets sans fuite de timing
 POURQUOI : Une comparaison naïve (==) est vulnérable aux attaques temporelles.
 CONTEXTE : Vérification de MACs, tokens, signatures, mots de passe hachés.

FONCTIONS PRINCIPALES :

<code>from cryptography.hazmat.primitives.constant_time import bytes_eq</code>	
<code>constant_time.bytes_eq(a, b)</code>	Compare a == b en temps constant → bool

EXEMPLES :

```
from cryptography.hazmat.primitives import constant_time
mac_recu = hmac_calcule_depuis_requete()
mac_attendu = hmac_stocke_en_base()
if constant_time.bytes_eq(mac_recu, mac_attendu):
    print("Authentifié")
```

[util] Exceptions – cryptography.exceptions

DESCRIPTION : Exceptions spécifiques du module cryptography
 POURQUOI : Distinguer les erreurs cryptographiques des erreurs génériques Python.
 CONTEXTE : Gestion d'erreurs dans tous les workflows de chiffrement / vérification.

EXCEPTIONS PRINCIPALES :

<code>from cryptography.exceptions import (</code>	
<code> InvalidSignature,</code>	Signature invalide (verify échoué)
<code> InvalidTag,</code>	Tag GCM/CCM invalide (intégrité)
<code> AlreadyFinalized,</code>	Objet déjà finalisé
<code> NotYetFinalized,</code>	Objet non encore finalisé
<code> UnsupportedAlgorithm,</code>	Algorithme non supporté

```

InvalidKey,                                Clé invalide pour l'opération
)
from cryptography.fernet import InvalidToken  Token Fernet invalide / expiré

```

EXEMPLES :

```

from cryptography.exceptions import InvalidSignature
from cryptography.fernet import InvalidToken

```

```

try:
    public_key.verify(sig, data, ...)
except InvalidSignature:
    print("Signature incorrecte")

```

```

try:
    f.decrypt(token)
except InvalidToken:
    print("Token invalide ou expiré")

```

[util] Sérialisation universelle – hazmat.primitives.serialization
--

DESCRIPTION : Encode et décode toutes les structures cryptographiques (clés, certs...)
 POURQUOI : Persister, transmettre ou charger des clés et certificats.
 CONTEXTE : Stockage fichier, transmission réseau, configuration, PEM/DER/SSH.

ENCODAGES :

serialization.Encoding.PEM	Base64 avec en-têtes -----BEGIN.....
serialization.Encoding.DER	Binaire ASN.1 brut
serialization.Encoding.OpenSSH	Format clé publique OpenSSH
serialization.Encoding.Raw	Octets bruts (courbes modernes)
serialization.Encoding.X962	Format point de courbe elliptique

FORMATS PRIVÉS :

serialization.PrivateFormat.PKCS8	PKCS#8 (standard moderne)
serialization.PrivateFormat.TraditionalOpenSSL	Format OpenSSL legacy
serialization.PrivateFormat.Raw	Octets bruts (Ed25519, X25519)
serialization.PrivateFormat.OpenSSH	Format clé privée OpenSSH

FORMATS PUBLICS :

serialization.PublicFormat.SubjectPublicKeyInfo	SPKI (standard)
serialization.PublicFormat.PKCS1	Format PKCS#1 (RSA uniquement)
serialization.PublicFormat.OpenSSH	Format clé publique SSH
serialization.PublicFormat.Raw	Octets bruts (Ed25519, X25519)
serialization.PublicFormat.UncompressedPoint	Point non compressé (EC)
serialization.PublicFormat.CompressedPoint	Point compressé (EC)

CHIFFREMENT DE CLÉ PRIVÉE :

serialization.BestAvailableEncryption(b"passphrase")	Chiffre avec AES-256-CBC
serialization.NoEncryption()	Pas de chiffrement

CHARGEMENT :

serialization.load_pem_private_key(data, password)	Charge PEM privé
serialization.load_pem_public_key(data)	Charge PEM public
serialization.load_der_private_key(data, password)	Charge DER privé
serialization.load_der_public_key(data)	Charge DER public
serialization.load_ssh_public_key(data)	Charge clé publique SSH

EXEMPLES :

```

# Sauvegarder une clé privée chiffrée
pem = priv.private_bytes(
    serialization.Encoding.PEM,
    serialization.PrivateFormat.PKCS8,
    serialization.BestAvailableEncryption(b"passphrase_secrete")
)
with open("cle_privée.pem", "wb") as f:
    f.write(pem)

# Recharger
with open("cle_privée.pem", "rb") as f:
    priv2 = serialization.load_pem_private_key(f.read(), password=b"passphrase_secrete")

```

=====

RÉCAPITULATIF DES IMPORTS PAR USAGE

```
=====

from cryptography.fernet import Fernet, MultiFernet, InvalidToken

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding as sym_padding

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.hashes import Hash

from cryptography.hazmat.primitives import hmac

from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from cryptography.hazmat.primitives.kdf.hkdf import HKDF, HKDFExpand

from cryptography.hazmat.primitives.asymmetric import rsa, ec, dh
from cryptography.hazmat.primitives.asymmetric import padding as asym_padding
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives.asymmetric.ed448 import Ed448PrivateKey
from cryptography.hazmat.primitives.asymmetric.x448 import X448PrivateKey

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.constant_time import bytes_eq
from cryptography.exceptions import InvalidSignature, InvalidTag

from cryptography import x509
from cryptography.x509.oid import NameOID, ExtendedKeyUsageOID

import os                                     # os.urandom() pour les aléas

=====
```